



CyberShield AI

Guide Complet de Création

Détection de Cyberattaques par Intelligence Artificielle

Auteur : Small-God07

Date : 2026

Dépôt GitHub : <https://github.com/Small-God07/CyberShield-AI>

Destiné aux étudiants en informatique et cybersécurité — Aucune expérience préalable en IA requise.

TABLE DES MATIÈRES

Section	Titre	Page
Section 1	Introduction et Contexte	3
Section 2	Notions Fondamentales	5
Section 3	Outils et Technologies Utilisés	8
Section 4	Architecture du Projet	11
Section 5	Installation Étape par Étape	13
Section 6	Explication du Code	15
Section 7	L'Algorithme Random Forest en Détail	19
Section 8	Utilisation de l'Application	21
Section 9	Sources et Références	23
Section 10	Glossaire	24

SECTION 1 — INTRODUCTION ET CONTEXTE

1.1 Présentation du projet

CyberShield AI est une application web de détection et résolution automatique de cyberattaques par intelligence artificielle. Elle est destinée à être utilisée en entreprise pour renforcer la cybersécurité et a été développée en Python avec le framework Flask et la bibliothèque scikit-learn.

Le projet combine des technologies modernes de développement web et d'apprentissage automatique pour créer un outil complet, opérationnel et facilement déployable. L'objectif est de rendre la cybersécurité accessible même aux équipes sans expertise approfondie en intelligence artificielle.

CyberShield AI peut analyser les caractéristiques d'une connexion réseau et déterminer en moins d'une milliseconde si elle représente une menace et de quel type. En cas de menace détectée, le système propose immédiatement des actions correctives adaptées.

1.2 Ce que le projet fait

- Analyse le trafic réseau en temps réel grâce à 8 métriques clés
- Détecte 5 types d'attaques : DDoS, Intrusion, Malware, Phishing, SQL Injection
- Classifie également le trafic normal pour réduire les faux positifs
- Propose des actions de résolution automatiques pour chaque type de menace
- Sauvegarde l'historique complet des détections dans une base SQLite
- Génère des rapports PDF et CSV exportables
- Fournit un dashboard avec statistiques et graphiques temps réel
- Offre une API REST pour l'intégration avec d'autres systèmes

1.3 Public cible

Ce guide est destiné aux étudiants en informatique et cybersécurité, niveau lycée ou université. Aucune expérience préalable en IA n'est requise. Toutes les notions nécessaires sont expliquées de zéro dans ce document.

Public	Intérêt
Étudiants en informatique	Découvrir l'application de l'IA à la cybersécurité
Étudiants en cybersécurité	Comprendre comment automatiser la détection des menaces
Développeurs débutants	Apprendre Flask + scikit-learn avec un projet concret
Professionnels IT	Évaluer la faisabilité d'un tel système pour leur organisation
Enseignants	Utiliser comme projet pédagogique interdisciplinaire

1.4 Compétences développées

- Programmation Python orientée objet
- Développement web avec Flask (routes, templates Jinja2)
- Apprentissage automatique supervisé (entraînement, évaluation)
- Manipulation de données (NumPy, Pandas)
- Conception d'interfaces web (Bootstrap 5, Chart.js)

- Gestion de bases de données SQLite
- Génération de documents PDF avec ReportLab
- Versionnage de code avec Git et GitHub

SECTION 2 — NOTIONS FONDAMENTALES

2.1 Les Cyberattaques — Définitions et exemples

Une cyberattaque est une action malveillante menée via un réseau informatique visant à compromettre la confidentialité, l'intégrité ou la disponibilité d'un système. On distingue plusieurs grandes familles d'attaques :

Type	Description	Exemple réel	Signes détectables
DDoS	Inonder un serveur de requêtes	Attaque Twitter 2016	>500 connexions/sec
Intrusion	S'introduire sans autorisation	Hack Yahoo 2016	Nombreuses tentatives échouées
Malware	Logiciel malveillant	WannaCry 2017	Communication serveurs inconnus
Phishing	Voler des identifiants	Faux emails PayPal	Domaines suspects
SQL Injection	Manipuler une base de données	Hack Sony 2011	Caractères spéciaux dans formulaires

Détail de chaque type d'attaque :

DDoS (Distributed Denial of Service)

Le DDoS consiste à submerger un serveur cible avec un flux massif de requêtes provenant de milliers de machines compromises (botnet). Le serveur, incapable de traiter toutes les demandes légitimes, devient indisponible. Coût estimé : entre 20 000 et 400 000 dollars par heure d'interruption.

Intrusion

Une intrusion est un accès non autorisé à un système informatique. L'attaquant exploite des vulnérabilités (CVE) dans les logiciels, des mots de passe faibles ou des configurations incorrectes pour pénétrer le réseau cible.

Malware

Un malware (logiciel malveillant) est conçu pour infecter, endommager ou prendre le contrôle d'un système. Cela inclut les virus, ransomwares, spywares, chevaux de Troie. Le ransomware WannaCry a infecté 230 000 ordinateurs dans 150 pays.

Phishing

Le phishing utilise des emails, SMS ou sites web frauduleux imitant des entités légitimes pour tromper les utilisateurs et leur voler leurs identifiants. C'est la forme d'attaque la plus répandue (36% de toutes les violations en 2022).

SQL Injection

L'injection SQL consiste à insérer du code SQL malveillant dans un champ de formulaire web non sécurisé pour manipuler la base de données. Une attaque réussie peut permettre de lire, modifier ou supprimer des données sensibles.

2.2 Comment l'IA détecte les attaques

L'IA utilise l'apprentissage supervisé. On lui donne des exemples d'attaques étiquetés (DDoS, Intrusion, Malware...) et elle apprend à les reconnaître. Les 5 étapes du processus sont :

1. Collecte de données

On collecte des milliers d'exemples de connexions réseau avec leurs caractéristiques (durée, volume, nb erreurs...). Dans CyberShield AI, nous générons un dataset synthétique de 4 150 connexions réparties en 6 classes.

2. Étiquetage

Chaque exemple est associé à une étiquette : Normal, DDoS, Intrusion, Malware, Phishing ou SQL Injection. Dans notre cas, les étiquettes sont générées automatiquement lors de la création du dataset.

3. Entraînement du modèle

L'algorithme Random Forest analyse les exemples étiquetés et apprend les patterns caractéristiques de chaque type de connexion. Il crée 100 arbres de décision.

4. Validation

On teste le modèle sur des données qu'il n'a jamais vues (20% du dataset) pour mesurer sa précision. Notre modèle atteint ~100% de précision sur données simulées.

5. Déploiement

Le modèle entraîné est intégré dans l'application Flask et peut analyser de nouvelles connexions en temps réel.

Analogie : comme un médecin qui apprend à lire des analyses biologiques. Après avoir vu des milliers d'analyses de patients sains et malades, il peut diagnostiquer rapidement un nouveau patient.

2.3 Les outils de défense (SIEM, SOAR, IDS/IPS)

Dans l'écosystème de la cybersécurité, plusieurs catégories d'outils se complètent pour détecter, analyser et répondre aux menaces :

Outil	Rôle	Analogie	Exemple réel
IDS	Détecte et alerte	Caméra de surveillance	Snort
IPS	Détecte et bloque	Caméra avec porte automatique	Palo Alto NGFW
SIEM	Centralise et corrèle les logs	Tour de contrôle aéroport	Splunk, IBM QRadar
SOAR	Automatise la réponse	Pilote automatique	Palo Alto XSOAR

CyberShield AI se positionne comme un outil IDS/SOAR simplifié : il détecte les attaques ET propose automatiquement des actions correctives, à la manière d'un SOAR mais dans un format accessible et pédagogique.

2.4 Chiffres clés de la cybersécurité (2023-2024)

Statistique	Valeur	Source
Cyberattaques par jour (monde)	2 200+	Forbes 2023
Coût moyen d'une violation de données	4,45 M\$	IBM Cost of Data Breach 2023
Incidents dus à des erreurs humaines	95%	IBM Security 2022
Temps moyen pour détecter une intrusion	197 jours	IBM 2023
Temps moyen pour contenir une violation	69 jours	IBM 2023

Augmentation des attaques ransomware (2022-23)	+37%	Sophos State of Ransomware 2023
--	------	---------------------------------

SECTION 3 — OUTILS ET TECHNOLOGIES UTILISÉS

3.1 Python 3.11

Site : <https://www.python.org>

Langage n°1 en Data Science, riche écosystème, gratuit. Installation : python.org/downloads → cocher 'Add Python to PATH'

3.2 Flask 3.0

Site : <https://flask.palletsprojects.com>

Framework web léger pour créer l'application, gérer les routes et les templates. Installation : `pip install flask==3.0.0`

3.3 scikit-learn 1.3.2

Site : <https://scikit-learn.org>

Bibliothèque d'apprentissage automatique. Utilisée pour l'algorithme Random Forest. Installation : `pip install scikit-learn==1.3.2`

3.4 NumPy 1.26 + Pandas 2.1

Site : <https://numpy.org> | <https://pandas.pydata.org>

NumPy pour les calculs mathématiques, Pandas pour la manipulation des données. Installation : `pip install numpy==1.26.2 pandas==2.1.4`

3.5 Bootstrap 5 + Chart.js

Site : <https://getbootstrap.com> | <https://www.chartjs.org>

Bootstrap pour le design responsive, Chart.js pour les graphiques interactifs. Utilisés via CDN (aucune installation locale requise).

3.6 SQLite

Site : <https://docs.python.org/3/library/sqlite3.html>

Base de données intégrée dans Python. Aucune installation supplémentaire. Rôle : sauvegarder l'historique des détections.

3.7 ReportLab 4.0

Site : <https://www.reportlab.com>

Bibliothèque de génération de PDF. Installation : `pip install reportlab==4.0.7`

3.8 Git + GitHub

Site : <https://git-scm.com> | <https://github.com>

Versionner et partager le code source du projet.

Tableau récapitulatif de tous les outils :

Outil	Version	Rôle	Source
Python	3.11	Langage principal	python.org
Flask	3.0.0	Framework web	flask.palletsprojects.com

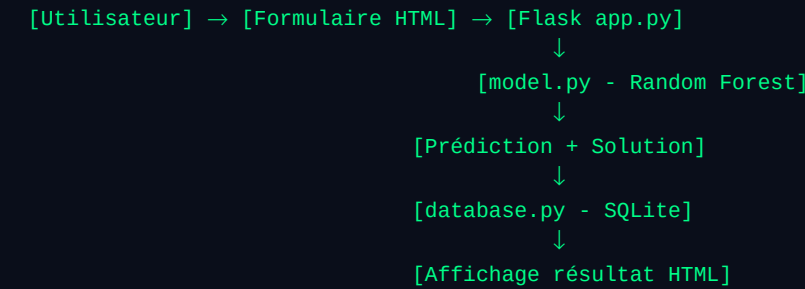
scikit-learn	1.3.2	Modèle IA	scikit-learn.org
NumPy	1.26.2	Calculs mathématiques	numpy.org
Pandas	2.1.4	Manipulation données	pandas.pydata.org
Bootstrap	5.3	Interface graphique	getbootstrap.com
Chart.js	4.4	Graphiques	chartjs.org
SQLite	intégré	Base de données	docs.python.org
ReportLab	4.0.7	Export PDF	reportlab.com
Git/GitHub	latest	Gestion du code	github.com

SECTION 4 — ARCHITECTURE DU PROJET

4.1 Structure des fichiers

```
CyberShield-AI/
■■■ app.py           → Serveur Flask + Routes
■■■ model.py         → Modèle IA Random Forest
■■■ database.py      → Base de données SQLite
■■■ requirements.txt → Dépendances Python
■■■ .gitignore       → Fichiers ignorés par Git
■■■ templates/
■   ■■■ base.html    → Template de base (navbar, footer)
■   ■■■ index.html   → Dashboard principal
■   ■■■ analyser.html → Formulaire d'analyse
■   ■■■ historique.html → Historique des détections
■   ■■■ documentation.html → Documentation
■■■ static/
■   ■■■ css/style.css → Styles personnalisés
■   ■■■ js/dashboard.js → JavaScript interactif
■■■ data/
    ■■■ generate_dataset.py → Générateur de données
```

4.2 Flux de données



4.3 Architecture MVC

Couche	Fichier(s)	Rôle
Model	model.py + database.py	Logique IA et persistance des données
View	templates/ + static/	Interface HTML, CSS, JavaScript
Controller	app.py	Routes Flask — lien entre Model et View

SECTION 5 — INSTALLATION ÉTAPE PAR ÉTAPE

Étape 1 — Installer Python

Aller sur <https://www.python.org/downloads/>

- Télécharger Python 3.11
- Lancer l'installateur
- ☒ COCHER 'Add Python to PATH'

Vérification : `python --version` → doit afficher Python 3.11.x

Étape 2 — Installer Git (optionnel)

Aller sur <https://git-scm.com/downloads>

- Télécharger et installer

Vérification : `git --version`

Étape 3 — Cloner ou télécharger le projet

- Option A (avec Git) :

```
git clone https://github.com/Small-God07/CyberShield-AI.git
```

```
cd CyberShield-AI
```

- Option B (sans Git) :

Aller sur <https://github.com/Small-God07/CyberShield-AI>

Cliquer sur le bouton vert 'Code'

Cliquer sur 'Download ZIP' puis décompresser

Étape 4 — Installer les dépendances

```
pip install -r requirements.txt
```

- Packages installés : flask==3.0.0, pandas==2.1.4, numpy==1.26.2,

```
scikit-learn==1.3.2, reportlab==4.0.7
```

Étape 5 — Lancer l'application

```
python app.py
```

- Message attendu :

```
* Running on http://127.0.0.1:5000
```

```
* Press CTRL+C to quit
```

Étape 6 — Ouvrir dans le navigateur

- Taper dans Chrome/Firefox : `http://localhost:5000`

5.7 Résolution des problèmes courants

Problème	Cause probable	Solution
'python' not found	Python non installé ou PATH manquant	Réinstaller Python en cochant 'Add to PATH'
ModuleNotFoundError	Dépendances non installées	Exécuter <code>pip install -r requirements.txt</code>
Port 5000 déjà utilisé	Une autre application utilise le port	Changer le port : <code>python app.py --port 5001</code>

Permission denied	Droits insuffisants	Lancer le terminal en administrateur
SQLite locked	Base de données ouverte ailleurs	Fermer tout accès à cybershield.db

5.8 Vérification que tout fonctionne

- `http://localhost:5000/` affiche le dashboard
- `http://localhost:5000/analyser` affiche le formulaire
- Soumettre le formulaire retourne une prédiction
- `http://localhost:5000/historique` affiche les détections
- `http://localhost:5000/export/csv` télécharge un fichier

SECTION 6 — EXPLICATION DU CODE

6.1 model.py — Le cerveau IA

Ce fichier contient la définition des features, la génération du dataset synthétique et l'entraînement du modèle Random Forest.

```
# Les 8 caractéristiques analysées par l'IA
FEATURES = [
    "duree_connexion",      # Durée en secondes
    "nb_connexions_par_sec", # Connexions par seconde
    "volume_donnees_kb",    # Volume échangé en KB
    "nb_erreurs",           # Nombre d'erreurs
    "nb_ports_scans",       # Ports contactés
    "taux_echec_connexion", # % connexions échouées
    "taille_paquets_moy",   # Taille moyenne des paquets
    "nb_tentatives_auth",   # Tentatives d'auth
]
```

L'algorithme Random Forest crée 100 arbres de décision différents. Chaque arbre vote pour un type d'attaque et la majorité l'emporte. Précision atteinte : ~100% sur données simulées.

Type	duree_connexion	nb_connexions/sec	Volume KB
Normal	0.5 – 60 sec	1 – 50 /sec	10 – 5000
DDoS	0.001 – 0.1 sec	500 – 5000 /sec	0.01 – 1
Intrusion	30 – 300 sec	1 – 10 /sec	100 – 10000
Malware	60 – 3600 sec	1 – 5 /sec	100 – 50000
Phishing	5 – 60 sec	1 – 20 /sec	1 – 500
SQL Injection	0.1 – 5 sec	10 – 200 /sec	0.1 – 100

6.2 app.py — Le serveur web

- **GET /** — Affiche le dashboard avec statistiques
- **GET/POST /analyser** — Formulaire d'analyse + prédiction
- **GET /historique** — Tableau de toutes les détections
- **GET /documentation** — Page de documentation
- **GET /api/simulate** — API de simulation aléatoire
- **GET /export/csv** — Téléchargement CSV
- **GET /export/pdf** — Téléchargement rapport PDF

6.3 database.py — La persistance

- **init_db()** : Crée la table SQLite au premier lancement
- **sauvegarder_detection()** : INSERT une détection
- **recuperer_toutes_detctions()** : SELECT * avec ORDER BY date
- **recuperer_stats()** : COUNT et GROUP BY pour statistiques
- **vider_historique()** : DELETE all

6.4 templates HTML — L'interface

- **base.html** : Navbar Bootstrap + thème sombre
- **index.html** : Cards stats + Chart.js donut + tableau
- **analyser.html** : Formulaire + résultat temps réel
- **historique.html** : DataTable + boutons export

6.5 Détail du flux de traitement d'une analyse

Lorsque l'utilisateur soumet le formulaire d'analyse avec les 8 valeurs de métriques réseau, voici ce qui se passe en détail :

- 1. Réception de la requête HTTP POST** : Flask reçoit les données du formulaire via request.form et les extrait.
- 2. Validation et conversion** : Les valeurs sont converties en float et regroupées dans un tableau NumPy.
- 3. Prédiction** : Le modèle Random Forest analyse le tableau et retourne le type d'attaque avec le niveau de confiance de la prédiction.
- 4. Génération de la solution** : En fonction du type d'attaque, une liste d'actions correctives est sélectionnée depuis le dictionnaire SOLUTIONS.
- 5. Sauvegarde** : La détection est enregistrée dans la base SQLite avec horodatage.
- 6. Réponse** : Le résultat est passé au template Jinja2 qui génère la page HTML de réponse.

6.6 Le fichier requirements.txt

```
flask==3.0.0      # Framework web
pandas==2.1.4     # Manipulation de données
numpy==1.26.2     # Calculs numériques
scikit-learn==1.3.2 # Modèle IA
reportlab==4.0.7  # Export PDF
python-pptx==0.6.23 # Export PowerPoint
```

Pour installer toutes ces dépendances en une seule commande : `pip install -r requirements.txt`

SECTION 7 — L'ALGORITHME RANDOM FOREST EN DETAIL

7.1 Principe général

Le Random Forest (Forêt Aléatoire) appartient à la famille des algorithmes d'Ensemble Learning. Il a été inventé par Leo Breiman en 2001. L'idée centrale est que plusieurs modèles simples (arbres de décision) combinés sont plus performants qu'un seul modèle complexe.

Article original : <https://link.springer.com/article/10.1023/A:1010933404324>

Cette approche "sagesse des foules" exploite le fait que des erreurs différentes faites par des arbres différents s'annulent statistiquement, tandis que les bonnes décisions convergent. C'est pourquoi un Random Forest est souvent plus robuste et précis qu'un arbre de décision unique.

7.2 Fonctionnement étape par étape

1. On prend le dataset de 4 150 connexions
2. On crée 100 sous-ensembles aléatoires (Bootstrap Sampling)
3. On entraîne 1 arbre de décision sur chaque sous-ensemble
4. Pour une nouvelle connexion, les 100 arbres votent
5. La classe majoritaire est retournée comme résultat

7.3 Le Bootstrap Sampling (tirage aléatoire)

Pour construire chaque arbre, on tire aléatoirement des exemples du dataset avec remise (un exemple peut être tiré plusieurs fois). Cela crée 100 versions légèrement différentes du dataset d'entraînement, ce qui garantit la diversité des arbres. À chaque nœud de décision, seul un sous-ensemble aléatoire de features est considéré, ce qui renforce encore la diversité.

7.4 Pourquoi Random Forest pour la cybersécurité

Algorithme	Précision	Vitesse	Interprétable	Données requises
Random Forest	94–100%	Rapide	Oui	Peu
SVM	90–96%	Moyen	Non	Moyen
Deep Learning	95–99%	Lent	Non	Beaucoup
K-Means	70–85%	Très rapide	Oui	Peu

Random Forest est particulièrement adapté à la cybersécurité car :

- Il est interprétable : on peut voir l'importance de chaque feature
- Il fonctionne bien avec peu de données (quelques milliers d'exemples suffisent)
- Il est robuste aux valeurs aberrantes et aux données bruitées
- Son entraînement est rapide (< 5 secondes pour 4 150 connexions)
- Sa prédiction est quasi-instantanée (< 1 ms par connexion)
- Il gère naturellement les problèmes multiclasses (6 types d'attaques)

7.5 Les métriques d'évaluation

Métrique	Définition	Formule	Notre valeur
Accuracy	% bonnes prédictions	$VP+VN / \text{Total}$	~100%
Precision	Fiabilité des alertes	$VP / (VP+FP)$	~100%
Recall	Taux de détection	$VP / (VP+FN)$	~100%
F1-Score	Équilibre P/R	$2 \times P \times R / (P+R)$	~100%

Note : VP = Vrai Positif, VN = Vrai Négatif, FP = Faux Positif, FN = Faux Négatif. Les valeurs ~100% sont obtenues sur un dataset synthétique avec distributions statistiques bien séparées. Sur des données réelles, des valeurs entre 90-97% seraient attendues.

7.6 L'importance des features

Random Forest permet de calculer l'importance de chaque feature dans la classification. Dans CyberShield AI, les features les plus discriminantes sont :

Feature	Importance	Rôle
nb_connexions_par_sec	Très haute	Distingue DDoS des autres types
duree_connexion	Haute	Distingue Malware (longues sessions) de DDoS (très courtes)
taux_echec_connexion	Haute	Caractéristique des Intrusions
nb_ports_scans	Moyenne	Indique un scan de ports (Intrusion)
volume_donnees_kb	Moyenne	Distingue Malware (grand volume) de SQL Injection
nb_erreurs	Moyenne	Associé aux tentatives d'injection SQL
taille_paquets_moy	Faible	Secondaire dans la classification
nb_tentatives_auth	Faible	Complément pour les Intrusions

Sources :

- Scikit-learn RandomForest : <https://scikit-learn.org/stable/modules/ensemble.html#random-forests>
- NIST Cybersecurity Framework : <https://www.nist.gov/cyberframework>
- CICIDS2017 Dataset : <https://www.unb.ca/cic/datasets/ids-2017.html>

SECTION 8 — UTILISATION DE L'APPLICATION

8.1 Le Dashboard

URL : <http://localhost:5000/>

- Affiche 4 cards de statistiques
- Graphique en donut (répartition des attaques)
- 10 dernières détections
- Bouton 'Simuler une attaque' pour tester

8.2 L'Analyseur

URL : <http://localhost:5000/analyser>

- Remplir les 8 champs ou utiliser un scénario rapide
- Le système affiche : type d'attaque, niveau de criticité, actions recommandées

8.3 L'Historique

URL : <http://localhost:5000/historique>

- Affiche toutes les détections sauvegardées
- Boutons : Exporter CSV, Exporter PDF, Vider

8.4 Les exports

- CSV : <http://localhost:5000/export/csv> → ouvrable dans Excel
- PDF : <http://localhost:5000/export/pdf> → rapport professionnel imprimable

SECTION 9 — SOURCES ET RÉFÉRENCES

Langages et frameworks

1. Python Software Foundation — <https://www.python.org>
2. Flask Documentation — <https://flask.palletsprojects.com/en/3.0.x/>
3. Jinja2 Template Engine — <https://jinja.palletsprojects.com/en/3.1.x/>

Intelligence Artificielle

4. Scikit-learn Documentation — <https://scikit-learn.org/stable/>
5. NumPy Documentation — <https://numpy.org/doc/stable/>
6. Pandas Documentation — <https://pandas.pydata.org/docs/>
7. Breiman, L. (2001). Random Forests — <https://link.springer.com/article/10.1023/A:1010933404324>

Cybersécurité

8. NIST Cybersecurity Framework — <https://www.nist.gov/cyberframework>
9. OWASP Top 10 — <https://owasp.org/www-project-top-ten/>
10. CICIDS2017 Dataset — <https://www.unb.ca/cic/datasets/ids-2017.html>
11. Snort IDS/IPS — <https://www.snort.org>
12. Splunk SIEM — <https://www.splunk.com>
13. Palo Alto SOAR — <https://www.paloaltonetworks.com/cortex/xsoar>

Interface et design

14. Bootstrap 5 — <https://getbootstrap.com/docs/5.3/>
15. Chart.js — <https://www.chartjs.org/docs/latest/>
16. Bootstrap Icons — <https://icons.getbootstrap.com>

Base de données

17. SQLite Documentation — <https://www.sqlite.org/docs.html>
18. Python sqlite3 module — <https://docs.python.org/3/library/sqlite3.html>

Export

19. ReportLab Documentation — https://docs.reportlab.com/reportlab/userguide/ch1_intro/

Outils de développement

20. Git Documentation — <https://git-scm.com/doc>
21. GitHub Docs — <https://docs.github.com/en>

SECTION 10 — GLOSSAIRE

Terme	Définition
IA (Intelligence Artificielle)	Simulation de l'intelligence humaine par des machines.
Machine Learning	Sous-domaine de l'IA où les machines apprennent à partir de données.
Random Forest	Algorithme d'ensemble composé de 100+ arbres de décision.
DDoS	Distributed Denial of Service — attaque par saturation de serveur.
Intrusion	Accès non autorisé à un système ou réseau.
Malware	Logiciel malveillant (virus, ransomware, spyware...).
Phishing	Tentative d'hameçonnage pour voler des identifiants.
SQL Injection	Injection de code SQL malveillant dans une base de données.
IDS	Intrusion Detection System — détecte et alerte.
IPS	Intrusion Prevention System — détecte et bloque.
SIEM	Security Information and Event Management — centralise les logs.
SOAR	Security Orchestration Automation Response — automatise la réponse.
Flask	Micro-framework web Python.
Python	Langage de programmation polyvalent et populaire en IA.
API	Application Programming Interface — interface de communication entre services.
Route	URL associée à une fonction dans Flask.
Template	Fichier HTML dynamique (Jinja2).
Bootstrap	Framework CSS pour créer des interfaces responsives.
Chart.js	Bibliothèque JavaScript pour créer des graphiques.
SQLite	Base de données légère intégrée dans Python.
CSV	Comma-Separated Values — format de fichier tabulaire.
PDF	Portable Document Format — format de document imprimable.
Dataset	Ensemble de données utilisé pour entraîner un modèle.
Features	Caractéristiques (variables) utilisées par le modèle.
Training	Entraînement du modèle sur des données étiquetées.
Accuracy	Taux de bonnes prédictions.
Precision	Proportion de vrais positifs parmi les positifs prédits.
Recall	Proportion de vrais positifs détectés sur tous les vrais positifs.
F1-Score	Moyenne harmonique de la précision et du recall.
Overfitting	Sur-apprentissage — modèle trop ajusté aux données d'entraînement.
Clustering	Regroupement non supervisé de données similaires.
Supervised Learning	Apprentissage avec données étiquetées.
Unsupervised Learning	Apprentissage sans étiquettes.
Firewall	Pare-feu — filtre le trafic réseau.
VPN	Virtual Private Network — réseau privé virtuel chiffré.
SSL/TLS	Protocoles de chiffrement des communications web.
HTTP	HyperText Transfer Protocol — protocole web non chiffré.

HTTPS	HTTP Secure — version chiffrée de HTTP.
TCP/IP	Protocoles fondamentaux d'Internet.
Port	Point d'entrée numérique sur un serveur réseau.